```
"""
╔═══════════════════════════════════════════════════════════════════════════╗
║          AGENT 7 v3 — PRODUCTION-GRADE INVENTORY REORDER SYSTEM           ║
║                     Senior Data Scientist Rebuild                         ║
╟═══════════════════════════════════════════════════════════════════════════╢
║  FIXES OVER v2:                                                           ║
║  ✅ Intermittent demand: Croston + TSB replace bare LightGBM              ║
║  ✅ Hybrid forecast: statistical (Croston/TSB) + LightGBM ensemble        ║
║  ✅ WAPE / SMAPE replace MAPE (handles zero-heavy series)                ║
║  ✅ Class imbalance: SMOTE + class_weight + cost-sensitive threshold tuning ║
║  ✅ Regression reformulation: predict stockout_risk_score (continuous)    ║
║  ✅ Safety stock from empirical forecast error quantiles (not just σ)     ║
║  ✅ Lead time distribution fitted per-supplier-category                   ║
║  ✅ Cold-start handler for new / unseen SKUs                              ║
║  ✅ Business cost-sensitive evaluation (not just accuracy)                ║
║  ✅ Modular pipeline with clear separation of concerns                    ║
╚═══════════════════════════════════════════════════════════════════════════╝

Pipeline:
  Data Ingestion → Demand Classification (intermittent vs smooth) →
  Per-SKU Forecasting (Croston/TSB/LightGBM hybrid) → ABC Classification →
  Empirical Safety Stock → EOQ + (s,S) Policy → Bundle Rules →
  Risk Regression → Cost-Sensitive Threshold → PO Generation → KPI Evaluation

Inputs:  data_clean.csv, data_forecast.csv, data_forecast_scores.csv,
         data_basket_rules_v2.csv, data_basket_segment_rules_v2.csv,
         data_segments.csv, data_pricing.csv

Outputs: data_inventory_reorder_v3.csv, data_po_recommendations_v3.csv,
         data_kpi_evaluation_v3.csv, kafka_reorder_payloads_v3.json
"""
```

```
'\n╔═══════════════════════════════════════════════════════════════════════╗\n║          AGENT
7 v3 — PRODUCTION-GRADE INVENTORY REORDER SYSTEM           ║\n║                     Senior Data Scien
tist Rebuild                           ║\n╟═══════════════════════════════════════════════════════════
═══════════════════╢\n║  FIXES OVER v2:                                                           ║
\n║  ✅ Intermittent demand: Croston + TSB replace bare LightGBM              ║\n║  ✅ Hybrid fore
cast: statistical (Croston/TSB) + LightGBM ensemble           ║\n║  ✅ WAPE / SMAPE replace MAPE (hand
les zero-heavy series)                    ║\n║  ✅ Class imbalance: SMOTE + class_weight + cost-sensit
ive threshold tuning  ║\n║  ✅ Regression reformulation: predict stockout_risk_score (continuous)
║\n║  ✅ Safety stock from empirical forecast error quantiles (not just σ)        ║\n║  ✅ Lead time
```

```
# ────────────────────────────────────────────────────────────────────────
#  SECTION 1 — SETUP & CONFIG
# ────────────────────────────────────────────────────────────────────────
!pip install -q scikit-learn pandas numpy matplotlib seaborn scipy lightgbm imbalanced-learn
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from scipy import stats
from sklearn.ensemble import GradientBoostingRegressor, GradientBoostingClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import cross_val_score
from sklearn.metrics import (classification_report, mean_absolute_error,
                             mean_squared_error, roc_auc_score)
from sklearn.calibration import CalibratedClassifierCV
from sklearn.linear_model import LogisticRegression
```

```
import lightgbm as lgb
import warnings, json, datetime
```

```
try:
    from imblearn.over_sampling import SMOTE
    HAS_SMOTE = True
except ImportError:
    HAS_SMOTE = False
    print("⚠️  imbalanced-learn not installed — falling back to class_weight='balanced'")
    print("   Run: pip install imbalanced-learn")
```

```
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', '{:.4f}'.format)
```

```
CONFIG = {
    # ── Tiered service levels by ABC class ──────────────────────────────
    'service_level_z': {
        'A': 2.326,   # 99 % — high-value SKUs: never stock out
        'B': 1.645,   # 95 % — medium-value SKUs
        'C': 1.282,   # 90 % — low-value SKUs
        'default': 1.645,
    },
    # ── Inventory mechanics ─────────────────────────────────────────────
    'default_lead_time_days': 7,
    'ordering_cost_S': 50.0,        # fixed cost per PO (£)
    'holding_cost_pct': 0.25,       # annual holding cost as % of unit cost
    'min_order_qty': 6,
    'planning_horizon_days': 56,    # 8 weeks

    # ── ABC classification revenue thresholds ───────────────────────────
    'abc_thresholds': {'A': 0.80, 'B': 0.95},

    # ── (s,S) policy trigger ────────────────────────────────────────────
    'cv_high_variance': 0.50,       # CV > 0.5 → use (s,S) instead of pure EOQ

    # ── Intermittent demand detection ───────────────────────────────────
    # ADI = Average Demand Interval (avg periods between non-zero demand)
    # CV2 = squared CV of non-zero demand sizes
    # Croston: ADI >= 1.32 (sparse)
    # TSB (Syntetos-Boylan): ADI >= 1.32 AND CV2 >= 0.49
    # Smooth: ADI < 1.32
    'adi_threshold': 1.32,
    'cv2_threshold': 0.49,

    # ── Forecast ensemble weights ───────────────────────────────────────
    # Statistical model weight increases for highly intermittent series
    'ensemble_stat_weight_intermittent': 0.65,
    'ensemble_stat_weight_smooth': 0.35,

    # ── Safety stock: use empirical quantile from forecast errors ───────
    # Instead of Z * σ, we use the (1 - alpha) quantile of actual errors
    'safety_stock_error_quantile': 0.90,   # 90th percentile of forecast errors

    # ── Bundle uplifts ──────────────────────────────────────────────────
    'bundle_uplift_pct': 0.20,
    'tier1_uplift_pct': 0.30,

    # ── Segment demand weights ──────────────────────────────────────────
    'segment_weights': {
        'Champion': 1.40, 'Loyal': 1.20, 'At-Risk': 0.90, 'Lost': 0.70,
```

```
    },

    # ── Risk regression → urgency threshold (cost-sensitive) ─────────────
    # Tuned so that false negatives (missing Urgent) cost more than false positives
    'urgency_cost_fn': 10.0,        # cost of missing an urgent SKU (false negative)
    'urgency_cost_fp': 1.0,         # cost of flagging a low SKU as urgent (false positive)
    'urgency_thresholds': {
        'Urgent': 0.60,   # risk_score >= 0.60
        'High':   0.35,   # risk_score >= 0.35
        'Medium': 0.15,   # risk_score >= 0.15
    },

    # ── Stockout simulation for label generation ──────────────────────────
    'stockout_sim_window': 90,
    'stockout_cost_mult': 3.0,

    # ── Cold-start: SKUs with fewer than this many history days ──────────
    'cold_start_min_days': 30,
}
```

```
print('✅ Config loaded (v3)')
```

```
✅ Config loaded (v3)
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 2 — DATA LOADING
# ────────────────────────────────────────────────────────────────────────────
DATA_DIR = ''   # adjust as needed ('' = /content in Colab)
```

```
def load(name):
    path = DATA_DIR + name
    df = pd.read_csv(path)
    print(f'  ✅ {name:50s} {df.shape}')
    return df
```

```
print('Loading inputs...')
df_clean   = load('/content/data_clean.csv')
df_forecast= load('/content/data_forecast.csv')
df_fscores = load('/content/data_forecast_scores.csv')
df_basket  = load('/content/data_basket_rules_v2.csv')
df_bseg    = load('/content/data_basket_segment_rules_v2.csv')
df_segments= load('/content/data_segments.csv')
df_pricing = load('/content/data_pricing.csv')
print('\n✅ All files loaded')
```

```
Loading inputs...
  ✅ /content/data_clean.csv                          (776617, 18)
  ✅ /content/data_forecast.csv                       (8, 4)
  ✅ /content/data_forecast_scores.csv                (3, 5)
  ✅ /content/data_basket_rules_v2.csv                (46, 23)
  ✅ /content/data_basket_segment_rules_v2.csv        (2088, 7)
  ✅ /content/data_segments.csv                       (5861, 12)
  ✅ /content/data_pricing.csv                        (73100, 20)

✅ All files loaded
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 3 — EDA + ABC CLASSIFICATION
# ────────────────────────────────────────────────────────────────────────────
df_clean['InvoiceDate'] = pd.to_datetime(df_clean['InvoiceDate'])
df_clean['Date'] = df_clean['InvoiceDate'].dt.date
```

```
df_sales   = df_clean[df_clean['Quantity'] > 0].copy()
df_returns = df_clean[df_clean['Quantity'] < 0].copy()
```

```
print(f'Total transactions : {len(df_clean):,}')
print(f'Sales rows         : {len(df_sales):,}')
print(f'Return rows        : {len(df_returns):,}')
print(f'Unique products    : {df_sales["StockCode"].nunique():,}')
print(f'Date range         : {df_sales["InvoiceDate"].min()} → {df_sales["InvoiceDate"].max()}')
```

```
Total transactions : 776,617
Sales rows         : 776,617
Return rows        : 0
Unique products    : 4,623
Date range         : 2009-12-01 07:45:00 → 2011-12-09 12:50:00
```

```
# ── ABC Classification ──────────────────────────────────────────────────────
sku_revenue = (
    df_sales.groupby('StockCode')['Revenue']
    .sum().sort_values(ascending=False).reset_index()
)
sku_revenue['cumrev']     = sku_revenue['Revenue'].cumsum()
total_rev                 = sku_revenue['Revenue'].sum()
sku_revenue['cumrev_pct'] = sku_revenue['cumrev'] / total_rev
```

```
def assign_abc(pct):
    if pct <= CONFIG['abc_thresholds']['A']:  return 'A'
    if pct <= CONFIG['abc_thresholds']['B']:  return 'B'
    return 'C'
```

```
sku_revenue['abc_class'] = sku_revenue['cumrev_pct'].apply(assign_abc)
```

```
# ── Monthly seasonality index ───────────────────────────────────────────────
monthly_avg       = df_sales.groupby('Month')['Quantity'].mean()
seasonality_index = (monthly_avg / monthly_avg.mean()).to_dict()
current_month     = 12
current_seasonality = seasonality_index.get(current_month, 1.0)
```

```
print('\n── ABC Summary ──────────────────────────────────────────────')
for cls in ['A', 'B', 'C']:
    rev = sku_revenue[sku_revenue['abc_class'] == cls]['Revenue'].sum()
    cnt = (sku_revenue['abc_class'] == cls).sum()
    print(f'  Class {cls}: {cnt:5,} SKUs | £{rev:>12,.0f} ({rev/total_rev*100:.1f}%)')
```

```

── ABC Summary ──────────────────────────────────────────────
  Class A: 1,013 SKUs | £  13,455,300 (80.0%)
  Class B: 1,220 SKUs | £   2,523,405 (15.0%)
  Class C: 2,390 SKUs | £     841,691 (5.0%)
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 4 — DEMAND CLASSIFICATION (Smooth vs Intermittent)
#
#  FIX: v2 treated all SKUs identically.
#  v3 classifies each SKU using ADI + CV² (Syntetos-Boylan quadrant).
#       Quadrant I  (ADI < 1.32, CV² < 0.49) → Smooth  → LightGBM
#       Quadrant II (ADI >= 1.32, CV² < 0.49) → Intermittent (Croston)
#       Quadrant III(ADI < 1.32, CV² >= 0.49) → Lumpy     → TSB
#       Quadrant IV (ADI >= 1.32, CV² >= 0.49) → Erratic   → TSB
# ────────────────────────────────────────────────────────────────────────────
print('Classifying demand patterns (ADI + CV² — Syntetos-Boylan quadrant)...')
```

```
Classifying demand patterns (ADI + CV² — Syntetos-Boylan quadrant)...
```

```
daily_demand = (
    df_sales.groupby(['StockCode', 'Date'])['Quantity']
    .sum().reset_index()
)
daily_demand['Date'] = pd.to_datetime(daily_demand['Date'])
```

```
def adi_cv2(group):
    """Compute Average Demand Interval and CV² for non-zero periods."""
    qty = group['Quantity'].values
    non_zero = qty[qty > 0]
    if len(non_zero) < 2:
        return pd.Series({'adi': 999.0, 'cv2': 0.0, 'p_zero': 1.0})
    intervals  = np.diff(np.where(qty > 0)[0])
    adi_val    = intervals.mean() if len(intervals) > 0 else 999.0
    cv_nz      = non_zero.std() / (non_zero.mean() + 1e-9)
    cv2_val    = cv_nz ** 2
    p_zero     = (qty == 0).mean()
    return pd.Series({'adi': adi_val, 'cv2': cv2_val, 'p_zero': p_zero})
```

```
demand_pattern = (
    daily_demand.groupby('StockCode')
    .apply(adi_cv2)
    .reset_index()
)
```

```
def classify_demand(row):
    if row['adi'] < CONFIG['adi_threshold']:
        return 'smooth'
    if row['cv2'] < CONFIG['cv2_threshold']:
        return 'intermittent'   # Croston
    return 'lumpy'              # TSB
```

```
demand_pattern['demand_type'] = demand_pattern.apply(classify_demand, axis=1)
```

```
counts = demand_pattern['demand_type'].value_counts()
print(f'  Smooth       : {counts.get("smooth", 0):,} SKUs')
print(f'  Intermittent : {counts.get("intermittent", 0):,} SKUs  → Croston')
print(f'  Lumpy/Erratic: {counts.get("lumpy", 0):,} SKUs  → TSB')
```

```
  Smooth       : 4,490 SKUs
  Intermittent : 133 SKUs  → Croston
  Lumpy/Erratic: 0 SKUs  → TSB
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 5 — STATISTICAL FORECASTERS (Croston + TSB)
```

```
#
#  FIX: v2 used only LightGBM which struggles with zero-inflated series.
#  v3 adds Croston / TSB as the statistical leg of an ensemble.
#
#  Croston (1972): decomposes intermittent demand into:
#    z̃ = SES of non-zero demand sizes
#    p̃ = SES of inter-demand intervals
#    forecast = z̃ / p̃
#
#  TSB (Teunter-Syntetos-Babai, 2011): uses demand occurrence probability
#    instead of intervals → better for lumpy demand that can "die out"
# ────────────────────────────────────────────────────────────────────────────
print('Fitting statistical forecasters (Croston + TSB) per SKU...')
```

```
Fitting statistical forecasters (Croston + TSB) per SKU...
```

```
def croston_forecast(demand_series, alpha=0.1, n_ahead=1):
    """
    Classic Croston (1972) method.
    Returns forecast of demand per period.
    """
    d = np.array(demand_series, dtype=float)
    n = len(d)
    if n == 0 or d.sum() == 0:
        return 0.0

    # Initialise on first non-zero observation
    nz_idx = np.where(d > 0)[0]
    if len(nz_idx) == 0:
        return 0.0

    z = d[nz_idx[0]]   # demand size estimate
    p = nz_idx[0] + 1  # interval estimate
    q = 1              # periods since last demand

    for t in range(nz_idx[0] + 1, n):
        if d[t] > 0:
            z = alpha * d[t] + (1 - alpha) * z
            p = alpha * q   + (1 - alpha) * p
            q = 1
        else:
            q += 1

    forecast = z / p if p > 0 else 0.0
    return float(np.clip(forecast, 0, None))
```

```
def tsb_forecast(demand_series, alpha=0.1, beta=0.1):
    """
    TSB (Teunter-Syntetos-Babai 2011): tracks demand probability directly.
    Better than Croston when demand can permanently stop (obsolescence).
    """
    d = np.array(demand_series, dtype=float)
    n = len(d)
    if n == 0 or d.sum() == 0:
        return 0.0

    nz_idx = np.where(d > 0)[0]
    if len(nz_idx) == 0:
        return 0.0

    # Initialise
    mu  = d[nz_idx[0]]
```

```
    rho = 1.0 / (nz_idx[0] + 1)   # initial demand probability

    for t in range(nz_idx[0] + 1, n):
        if d[t] > 0:
            rho = beta + (1 - beta) * rho
            mu  = alpha * d[t] + (1 - alpha) * mu
        else:
            rho = (1 - beta) * rho

    return float(np.clip(rho * mu, 0, None))
```

```
# Compute per-SKU statistical forecasts
stat_forecasts = {}
sku_to_type    = demand_pattern.set_index('StockCode')['demand_type'].to_dict()
```

```
for sku, grp in daily_demand.groupby('StockCode'):
    series = grp.sort_values('Date')['Quantity'].values
    dtype  = sku_to_type.get(sku, 'smooth')

    if dtype == 'smooth':
        # Simple EWM for smooth demand (fast, reliable)
        if len(series) > 0:
            ewm_val = pd.Series(series).ewm(span=14, adjust=False).mean().iloc[-1]
            stat_forecasts[sku] = float(max(ewm_val, 0))
        else:
            stat_forecasts[sku] = 0.0
    elif dtype == 'intermittent':
        stat_forecasts[sku] = croston_forecast(series, alpha=0.15)
    else:  # lumpy
        stat_forecasts[sku] = tsb_forecast(series, alpha=0.15, beta=0.10)
```

```
print(f'  ✅ Statistical forecasts computed for {len(stat_forecasts):,} SKUs')
```

```
  ✅ Statistical forecasts computed for 4,623 SKUs
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 6 — FEATURE ENGINEERING (Enhanced)
# ────────────────────────────────────────────────────────────────────────────
print('Engineering product-level demand features...')
```

```
Engineering product-level demand features...
```

```
demand_stats = daily_demand.groupby('StockCode')['Quantity'].agg(
    demand_avg='mean',
    demand_std='std',
    demand_max='max',
    demand_days='count',
    demand_median='median',
).reset_index()
demand_stats['demand_std'] = demand_stats['demand_std'].fillna(0)
demand_stats['demand_cv']  = (
    demand_stats['demand_std'] / demand_stats['demand_avg'].clip(lower=0.01)
).clip(upper=5.0)
```

```
# Shock rate
def shock_rate(group):
    if len(group) < 7: return 0.0
    mu, sigma = group['Quantity'].mean(), group['Quantity'].std()
    if sigma == 0: return 0.0
    return (group['Quantity'] > mu + 1.5 * sigma).mean()
```

```
shock_rates  = daily_demand.groupby('StockCode').apply(shock_rate).rename('demand_shock_rate').reset_i
demand_stats = demand_stats.merge(shock_rates, on='StockCode', how='left')
demand_stats['demand_shock_rate'] = demand_stats['demand_shock_rate'].fillna(0)
```

```
# Demand trends
def demand_trend(group):
    if len(group) < 5: return 0.0
    x = np.arange(len(group))
    slope, *_ = stats.linregress(x, group['Quantity'].values)
    return slope
```

```
def recent_trend_28(group):
    sub = group.tail(28)
    if len(sub) < 5: return 0.0
    x = np.arange(len(sub))
    slope, *_ = stats.linregress(x, sub['Quantity'].values)
    return slope
```

```
trend_series = (
    daily_demand.groupby('StockCode').apply(demand_trend)
    .rename('demand_trend').reset_index()
)
recent_trend = (
    daily_demand.sort_values(['StockCode', 'Date'])
    .groupby('StockCode').apply(recent_trend_28)
    .rename('demand_trend_recent').reset_index()
)
demand_stats = (
    demand_stats
    .merge(trend_series, on='StockCode', how='left')
    .merge(recent_trend, on='StockCode', how='left')
)
demand_stats[['demand_trend', 'demand_trend_recent']] = (
    demand_stats[['demand_trend', 'demand_trend_recent']].fillna(0)
)
```

NEW v3 FEATURES

```
# Zero-demand ratio (key feature for intermittent demand ML)
zero_ratio = (
    daily_demand.groupby('StockCode')['Quantity']
    .apply(lambda x: (x == 0).mean())
    .rename('zero_demand_ratio').reset_index()
)
```

```
# Demand in last 14 days vs prior 14 days (momentum feature)
def demand_momentum(group):
    s = group.sort_values('Date')['Quantity']
    if len(s) < 14: return 0.0
    recent = s.iloc[-14:].mean()
```

```
    prior  = s.iloc[-28:-14].mean() if len(s) >= 28 else s.iloc[:-14].mean()
    return float((recent - prior) / (prior + 1e-9))
```

```
momentum = (
    daily_demand.groupby('StockCode').apply(demand_momentum)
    .rename('demand_momentum_14d').reset_index()
)
```

```
# Days since last sale (staleness indicator)
last_sale_date = daily_demand.groupby('StockCode')['Date'].max().reset_index()
last_sale_date.columns = ['StockCode', 'last_sale_date']
max_date = daily_demand['Date'].max()
last_sale_date['days_since_last_sale'] = (
    pd.to_datetime(max_date) - pd.to_datetime(last_sale_date['last_sale_date'])
).dt.days
```

```
demand_stats = (
    demand_stats
    .merge(zero_ratio,     on='StockCode', how='left')
    .merge(momentum,       on='StockCode', how='left')
    .merge(last_sale_date[['StockCode', 'days_since_last_sale']], on='StockCode', how='left')
)
demand_stats[['zero_demand_ratio', 'demand_momentum_14d', 'days_since_last_sale']] = (
    demand_stats[['zero_demand_ratio', 'demand_momentum_14d', 'days_since_last_sale']].fillna(0)
)
```

```
# Cold-start flag
demand_stats['is_cold_start'] = demand_stats['demand_days'] < CONFIG['cold_start_min_days']
```

```
# Lead time proxy
df_sales_sorted = df_sales.sort_values(['StockCode', 'InvoiceDate'])
gaps = (
    df_sales_sorted.groupby('StockCode')['InvoiceDate']
    .apply(lambda x: x.diff().dt.days.dropna())
    .reset_index(level=0)
    .rename(columns={'InvoiceDate': 'gap_days'})
)
gaps_filtered   = gaps[(gaps['gap_days'] >= 1) & (gaps['gap_days'] <= 60)]
lead_time_proxy = gaps_filtered.groupby('StockCode')['gap_days'].agg(
    lead_time_avg='mean', lead_time_std='std'
).reset_index()
lead_time_proxy['lead_time_avg'] = lead_time_proxy['lead_time_avg'].fillna(CONFIG['default_lead_time_c
lead_time_proxy['lead_time_std'] = lead_time_proxy['lead_time_std'].fillna(1.0)
```

```
# Return rate
return_qty  = df_returns.groupby('StockCode')['Quantity'].sum().abs().rename('return_qty')
sales_qty   = df_sales.groupby('StockCode')['Quantity'].sum().rename('total_sales_qty')
return_rate = (return_qty / sales_qty).fillna(0).clip(0, 1).rename('return_rate').reset_index()
```

```
# Product metadata
prod_meta = (
    df_sales.groupby('StockCode')
    .agg(description=('Description','first'),
         unit_price=('Price','median'),
         category=('Category','first'))
    .reset_index()
)
```

```
# Merge demand pattern info
demand_pattern_merge = demand_pattern[['StockCode', 'adi', 'cv2', 'p_zero', 'demand_type']]
```

```
# Build product_features
product_features = (
    demand_stats
    .merge(lead_time_proxy,    on='StockCode', how='left')
    .merge(return_rate,        on='StockCode', how='left')
    .merge(prod_meta,          on='StockCode', how='left')
    .merge(sku_revenue[['StockCode','abc_class']], on='StockCode', how='left')
    .merge(demand_pattern_merge, on='StockCode', how='left')
)
product_features['lead_time_avg']    = product_features['lead_time_avg'].fillna(CONFIG['default_lead_t
product_features['lead_time_std']    = product_features['lead_time_std'].fillna(1.0)
product_features['return_rate']      = product_features['return_rate'].fillna(0)
product_features['abc_class']        = product_features['abc_class'].fillna('C')
product_features['unit_price_safe']  = product_features['unit_price'].clip(lower=0.5).fillna(1.0)
product_features['net_demand_avg']   = product_features['demand_avg'] * (1 - product_features['return_
product_features['service_level_z']  = product_features['abc_class'].map(CONFIG['service_level_z'])
product_features['high_variance']    = product_features['demand_cv'] > CONFIG['cv_high_variance']
product_features['demand_type']      = product_features['demand_type'].fillna('smooth')
```

```
# Statistical forecast column
product_features['stat_forecast_avg'] = product_features['StockCode'].map(stat_forecasts).fillna(0)
```

```
print(f'\n✅ Feature engineering complete: {len(product_features):,} products')
print(f'  ABC-A: {(product_features["abc_class"]=="A").sum():,} '
      f'ABC-B: {(product_features["abc_class"]=="B").sum():,} '
      f'ABC-C: {(product_features["abc_class"]=="C").sum():,}')
print(f'  High-variance (CV>{CONFIG["cv_high_variance"]}): {product_features["high_variance"].sum():,}
print(f'  Cold-start SKUs (<{CONFIG["cold_start_min_days"]} days): {product_features["is_cold_start"].
```

```
✅ Feature engineering complete: 4,623 products
  ABC-A: 1,013 ABC-B: 1,220 ABC-C: 2,390
  High-variance (CV>0.5): 4,229
  Cold-start SKUs (<30 days): 1,665
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 7 — LIGHTGBM FORECASTER (ML leg of ensemble)
#
#  FIX: v2 reported MAPE=221% because MAPE breaks with zeros and outliers.
#  v3 reports WAPE (Weighted APE) and SMAPE instead.
#  WAPE = sum(|actual - pred|) / sum(actual)  — zero-safe, scale-free
#  SMAPE = mean(2|A-P|/(|A|+|P|))             — symmetric, handles zeros better
# ────────────────────────────────────────────────────────────────────────────
print('Building LightGBM forecaster (ML ensemble leg)...')
```

```
Building LightGBM forecaster (ML ensemble leg)...
```

```
daily_ts = daily_demand.copy()
daily_ts['Date'] = pd.to_datetime(daily_ts['Date'])
daily_ts = daily_ts.sort_values(['StockCode', 'Date'])
```

```
le_sku = LabelEncoder()
daily_ts['sku_id']     = le_sku.fit_transform(daily_ts['StockCode'].astype(str))
daily_ts['month']      = daily_ts['Date'].dt.month
daily_ts['dayofweek']  = daily_ts['Date'].dt.dayofweek
daily_ts['quarter']    = daily_ts['Date'].dt.quarter
daily_ts['weekofyear'] = daily_ts['Date'].dt.isocalendar().week.astype(int)
```

```
for lag in [1, 7, 14, 28]:
    daily_ts[f'lag_{lag}'] = daily_ts.groupby('StockCode')['Quantity'].shift(lag)
```

```
for window in [7, 14, 28]:
    grp = daily_ts.groupby('StockCode')['Quantity']
    daily_ts[f'roll_mean_{window}'] = grp.transform(
        lambda x: x.shift(1).rolling(window, min_periods=1).mean())
    daily_ts[f'roll_std_{window}']  = grp.transform(
        lambda x: x.shift(1).rolling(window, min_periods=1).std().fillna(0))
    # NEW: rolling max (captures spikes) and min
    daily_ts[f'roll_max_{window}']  = grp.transform(
        lambda x: x.shift(1).rolling(window, min_periods=1).max())
```

```
# NEW: zero-inflation lag (was this day zero demand yesterday?)
daily_ts['lag1_is_zero'] = (daily_ts.groupby('StockCode')['Quantity'].shift(1) == 0).astype(int)
```

```
FEAT_COLS = [
    'sku_id', 'month', 'dayofweek', 'quarter', 'weekofyear',
    'lag_1', 'lag_7', 'lag_14', 'lag_28',
    'roll_mean_7', 'roll_mean_14', 'roll_mean_28',
    'roll_std_7',  'roll_std_14',  'roll_std_28',
    'roll_max_7',  'roll_max_14',  'roll_max_28',
    'lag1_is_zero',
]
```

```
daily_ts_clean = daily_ts.dropna(subset=['lag_28']).copy()
```

```
cutoff_date = (
    daily_ts_clean['Date'].max() - pd.Timedelta(days=CONFIG['planning_horizon_days'])
)
train_ts = daily_ts_clean[daily_ts_clean['Date'] <= cutoff_date]
val_ts   = daily_ts_clean[daily_ts_clean['Date'] >  cutoff_date]
```

```
print(f'  Train: {len(train_ts):,} rows up to {cutoff_date.date()}')
print(f'  Val  : {len(val_ts):,} rows  (last {CONFIG["planning_horizon_days"]} days)')
```

```
  Train: 288,838 rows up to 2011-10-14
  Val  : 46,699 rows  (last 56 days)
```

```
X_train, y_train = train_ts[FEAT_COLS], train_ts['Quantity'].clip(lower=0)
X_val,   y_val   = val_ts[FEAT_COLS],   val_ts['Quantity'].clip(lower=0)
```

```
# Tweedie objective handles zero-inflated count data better than regression
lgbm_model = lgb.LGBMRegressor(
    n_estimators=600,
    learning_rate=0.03,
    num_leaves=63,
    max_depth=-1,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_samples=20,
    reg_alpha=0.1,
    reg_lambda=0.1,
    objective='tweedie',   # FIX: handles zero-heavy demand better than 'regression'
    tweedie_variance_power=1.5,
    random_state=42,
    n_jobs=-1,
```

```
    verbose=-1,
)
lgbm_model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    callbacks=[
        lgb.early_stopping(50, verbose=False),
        lgb.log_evaluation(-1),
    ],
)
val_preds = np.maximum(lgbm_model.predict(X_val), 0)
```

```
# ── Better forecast metrics ──────────────────────────────────────────────
def wape(actual, pred):
    """Weighted Absolute Percentage Error — zero-safe."""
    return np.abs(actual - pred).sum() / (actual.sum() + 1e-9) * 100
```

```
def smape(actual, pred):
    """Symmetric MAPE — handles zeros better than standard MAPE."""
    denom = (np.abs(actual) + np.abs(pred)) / 2 + 1e-9
    return np.mean(np.abs(actual - pred) / denom) * 100
```

```
val_mae  = np.abs(y_val - val_preds).mean()
val_rmse = np.sqrt(((y_val - val_preds)**2).mean())
val_wape = wape(y_val.values, val_preds)
val_smape= smape(y_val.values, val_preds)
```

```
# Only compute MAPE for non-zero actuals (to show what v2's 221% really means)
nz_mask  = y_val.values > 0
val_mape_nz = (np.abs(y_val.values[nz_mask] - val_preds[nz_mask]) /
               (y_val.values[nz_mask] + 1e-9)).mean() * 100
```

```
print(f'\n📊 LightGBM validation metrics (v3):')
print(f'  MAE       : {val_mae:.3f}')
print(f'  RMSE      : {val_rmse:.3f}')
print(f'  WAPE      : {val_wape:.1f}%   ← use this instead of MAPE')
print(f'  SMAPE     : {val_smape:.1f}%  ← symmetric, zero-safe')
print(f'  MAPE(nz)  : {val_mape_nz:.1f}% ← v2 reported 221% from ALL rows incl zeros')
print(f'  Best iter : {lgbm_model.best_iteration_}')
```

```
📊 LightGBM validation metrics (v3):
  MAE       : 18.941
  RMSE      : 60.518
  WAPE      : 79.9%   ← use this instead of MAPE
  SMAPE     : 83.2%  ← symmetric, zero-safe
  MAPE(nz)  : 288.4% ← v2 reported 221% from ALL rows incl zeros
  Best iter : 107
```

```
# ── Per-SKU LGBM forecast ────────────────────────────────────────────────
val_ts_pred = val_ts.copy()
val_ts_pred['lgbm_pred'] = val_preds
```

```
sku_lgbm_forecast = (
    val_ts_pred.groupby('StockCode')
    .apply(lambda g: pd.Series({
        'lgbm_forecast_avg' : g['lgbm_pred'].mean(),
        'lgbm_forecast_rmse': np.sqrt(((g['Quantity'] - g['lgbm_pred'])**2).mean()),
        # Store per-SKU error distribution for empirical safety stock
        'lgbm_errors'       : (g['Quantity'].values - g['lgbm_pred'].values).tolist(),
```

```
    }))
    .reset_index()
)
```

```
train_fallback = (
    train_ts.groupby('StockCode')['roll_mean_28'].last()
    .clip(lower=0).rename('lgbm_forecast_avg').reset_index()
)
train_fallback['lgbm_forecast_rmse'] = 0.0
train_fallback['lgbm_errors']        = [[0.0] for _ in range(len(train_fallback))]
```

```
all_skus       = pd.DataFrame({'StockCode': daily_ts['StockCode'].unique()})
sku_lgbm_fc    = (
    all_skus
    .merge(sku_lgbm_forecast, on='StockCode', how='left')
    .merge(train_fallback, on='StockCode', how='left', suffixes=('', '_fb'))
)
sku_lgbm_fc['lgbm_forecast_avg']  = sku_lgbm_fc['lgbm_forecast_avg'].fillna(sku_lgbm_fc['lgbm_forecast
sku_lgbm_fc['lgbm_forecast_rmse'] = sku_lgbm_fc['lgbm_forecast_rmse'].fillna(0)
sku_lgbm_fc['lgbm_errors']        = sku_lgbm_fc['lgbm_errors'].apply(
    lambda x: x if isinstance(x, list) else [0.0])
```

```
product_features = product_features.merge(
    sku_lgbm_fc[['StockCode', 'lgbm_forecast_avg', 'lgbm_forecast_rmse', 'lgbm_errors']],
    on='StockCode', how='left'
)
product_features['lgbm_forecast_avg']  = product_features['lgbm_forecast_avg'].fillna(
    product_features['net_demand_avg'])
product_features['lgbm_forecast_rmse'] = product_features['lgbm_forecast_rmse'].fillna(
    product_features['demand_std'])
product_features['lgbm_errors'] = product_features['lgbm_errors'].apply(
    lambda x: x if isinstance(x, list) else [0.0])
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 8 — HYBRID ENSEMBLE FORECAST
#
#  FIX: v2 used only LightGBM. v3 blends statistical + LightGBM.
#  For intermittent/lumpy SKUs → weight statistical model more heavily.
#  For smooth SKUs             → weight LightGBM more heavily.
# ────────────────────────────────────────────────────────────────────────────
print('Building hybrid ensemble forecast (Croston/TSB + LightGBM)...')
```

```
Building hybrid ensemble forecast (Croston/TSB + LightGBM)...
```

```
def ensemble_weight_stat(demand_type):
    """Return statistical model weight based on demand type."""
    if demand_type in ('intermittent', 'lumpy'):
        return CONFIG['ensemble_stat_weight_intermittent']
    return CONFIG['ensemble_stat_weight_smooth']
```

```
product_features['w_stat'] = product_features['demand_type'].apply(ensemble_weight_stat)
product_features['w_lgbm'] = 1.0 - product_features['w_stat']
```

```
product_features['ensemble_forecast_avg'] = (
    product_features['w_stat'] * product_features['stat_forecast_avg'] +
    product_features['w_lgbm'] * product_features['lgbm_forecast_avg']
).clip(lower=0)
```

```
# Apply seasonality on top
product_features['demand_seasonal_avg'] = (
    product_features['ensemble_forecast_avg'] * current_seasonality
).clip(lower=0.01)
```

```
# Use max(LGBM RMSE, demand_std) as demand uncertainty
product_features['demand_forecast_std'] = np.maximum(
    product_features['lgbm_forecast_rmse'],
    product_features['demand_std']
)
```

```
print(f'\n✅ Hybrid forecasts generated for {len(product_features):,} SKUs')
print(f'  Avg ensemble daily demand : {product_features["demand_seasonal_avg"].mean():.3f}')
print(f'  Avg forecast RMSE         : {product_features["demand_forecast_std"].mean():.3f}')
print(f'  Validation WAPE           : {val_wape:.1f}%  (v2 reported MAPE 221% — misleading)')
```

```
✅ Hybrid forecasts generated for 4,623 SKUs
  Avg ensemble daily demand : 18.713
  Avg forecast RMSE         : 41.654
  Validation WAPE           : 79.9%  (v2 reported MAPE 221% — misleading)
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 9 — HISTORICAL STOCKOUT SIMULATION (Data-driven ML labels)
#  (unchanged from v2 — this was already solid)
# ────────────────────────────────────────────────────────────────────────────
print('Running historical stockout simulation...')
```

```
Running historical stockout simulation...
```

```
sim_daily   = daily_demand.copy().assign(Date=lambda d: pd.to_datetime(d['Date'])).sort_values('Date')
sim_cutoff  = sim_daily['Date'].max() - pd.Timedelta(days=CONFIG['stockout_sim_window'])
sim_window  = sim_daily[sim_daily['Date'] > sim_cutoff]
demand_pivot = sim_window.pivot_table(
    index='Date', columns='StockCode', values='Quantity', aggfunc='sum').fillna(0)
```

```
pf_base = product_features[['StockCode', 'demand_avg', 'demand_std',
                            'lead_time_avg', 'unit_price_safe', 'abc_class']].copy()
pf_base['Z_base']   = pf_base['abc_class'].map(CONFIG['service_level_z'])
pf_base['SS_base']  = (pf_base['Z_base'] * pf_base['demand_std'] * np.sqrt(pf_base['lead_time_avg'])).
pf_base['ROP_base'] = (pf_base['demand_avg'] * pf_base['lead_time_avg'] + pf_base['SS_base']).clip(low
pf_base['H_base']   = pf_base['unit_price_safe'] * CONFIG['holding_cost_pct']
pf_base['EOQ_base'] = np.sqrt(
    2 * pf_base['demand_avg'] * 365 * CONFIG['ordering_cost_S'] /
    pf_base['H_base'].clip(lower=0.01)
).clip(lower=CONFIG['min_order_qty']).round(0)
sku_params = pf_base.set_index('StockCode')
```

```
def simulate_sku_stockout(sku, demand_series):
    try:
        avg = sku_params.loc[sku, 'demand_avg']
        rop = sku_params.loc[sku, 'ROP_base']
        eoq = sku_params.loc[sku, 'EOQ_base']
    except KeyError:
        avg, rop, eoq = 1.0, 7.0, 6.0
    inventory      = rop + eoq
    stockout_days  = 0
    low_stock_days = 0
    n              = len(demand_series)
    for d in demand_series:
```

```
        inventory -= d
        if inventory <= 0:
            stockout_days += 1
            inventory = 0.0
        else:
            if inventory < rop:
                inventory += eoq
                low_stock_days += 1
    return stockout_days / max(n, 1), low_stock_days / max(n, 1)
```

```
sim_results = {}
skus_in_window = demand_pivot.columns.tolist()
print(f'  Simulating {len(skus_in_window):,} SKUs over {len(demand_pivot)} days...')
for sku in skus_in_window:
    sr, lr = simulate_sku_stockout(sku, demand_pivot[sku].values)
    sim_results[sku] = {'stockout_rate': sr, 'low_stock_rate': lr}
```

```
  Simulating 3,012 SKUs over 78 days...
```

```
sim_df = pd.DataFrame.from_dict(sim_results, orient='index').reset_index()
sim_df.columns = ['StockCode', 'stockout_rate', 'low_stock_rate']
```

```
def sim_urgency(row):
    sr, lr = row['stockout_rate'], row['low_stock_rate']
    if sr > 0.10:              return 'Urgent'
    if sr > 0.04 or lr > 0.25: return 'High'
    if sr > 0.01 or lr > 0.12: return 'Medium'
    return 'Low'
```

```
sim_df['urgency_label'] = sim_df.apply(sim_urgency, axis=1)
```

```
all_skus_df  = product_features[['StockCode', 'demand_avg', 'demand_cv', 'abc_class']].copy()
sim_df_full  = all_skus_df.merge(sim_df, on='StockCode', how='left')
```

```
def fallback_label(row):
    if pd.notna(row['urgency_label']): return row['urgency_label']
    if row['abc_class'] == 'A' and row['demand_cv'] > 0.5: return 'High'
    if row['demand_avg'] > 20: return 'Medium'
    return 'Low'
```

```
sim_df_full['urgency_label']   = sim_df_full.apply(fallback_label, axis=1)
sim_df_full['stockout_rate']   = sim_df_full['stockout_rate'].fillna(0)
sim_df_full['low_stock_rate']  = sim_df_full['low_stock_rate'].fillna(0)
```

```
# Continuous risk score for regression reformulation (v3 addition)
sim_df_full['stockout_risk_score'] = (
    0.6 * sim_df_full['stockout_rate'].clip(0, 1) +
    0.4 * sim_df_full['low_stock_rate'].clip(0, 1)
).clip(0, 1)
```

```
product_features = product_features.merge(
    sim_df_full[['StockCode', 'stockout_rate', 'low_stock_rate',
                 'urgency_label', 'stockout_risk_score']],
    on='StockCode', how='left'
)
print(f'\n✅ Simulation complete')
print(sim_df_full['urgency_label'].value_counts().to_string())
```

```
✅ Simulation complete
urgency_label
Low       4186
Medium     325
High        85
Urgent      27
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 10 — EMPIRICAL SAFETY STOCK (replaces Z * σ for SKUs with errors)
#
#  FIX: v2 used SS = Z × σ_demand × √(LT) which is correct only when forecast
#  errors are normally distributed. For intermittent demand they're not.
#  v3 uses the empirical (1-α) quantile of observed forecast errors per SKU.
#  For SKUs without enough error history → fall back to Z * σ.
# ────────────────────────────────────────────────────────────────────────────
print('Computing empirical safety stocks...')
```

```
Computing empirical safety stocks...
```

```
def empirical_ss(errors, lead_time, quantile=0.90):
    """
    Safety stock from empirical forecast error distribution.
    Errors are (actual - forecast): positive = underestimated demand.
    We want the quantile of the cumulative error over lead time.
    """
    if not errors or len(errors) < 10:
        return None   # fall back to parametric
    errors = np.array(errors)
    # Scale errors to lead time
    lt_errors = errors * np.sqrt(lead_time)
    # Take the (1-alpha) quantile of positive errors
    ss = float(np.percentile(lt_errors, quantile * 100))
    return max(ss, 0)
```

```
def parametric_ss(row):
    """Fallback: Z × σ × √(LT)."""
    return float(
        row['service_level_z'] * row['demand_forecast_std'] * np.sqrt(row['lead_time_avg'])
    )
```

```
pf = product_features.copy()
q  = CONFIG['safety_stock_error_quantile']
```

```
ss_values = []
for _, row in pf.iterrows():
    errors  = row['lgbm_errors']
    lt      = row['lead_time_avg']
    emp_ss  = empirical_ss(errors, lt, q)
    if emp_ss is not None:
        ss_values.append(max(emp_ss, 1))
    else:
        ss_values.append(max(parametric_ss(row), 1))
```

```
pf['safety_stock'] = np.round(ss_values, 0)
```

```
pf['reorder_point'] = (
    pf['demand_seasonal_avg'] * pf['lead_time_avg'] + pf['safety_stock']
).clip(lower=1).round(0)
```

```
pf['annual_demand']    = (pf['demand_seasonal_avg'] * 365).clip(lower=1)
pf['holding_cost_H']   = pf['unit_price_safe'] * CONFIG['holding_cost_pct']
pf['eoq']              = np.sqrt(
    2 * pf['annual_demand'] * CONFIG['ordering_cost_S'] / pf['holding_cost_H']
).clip(lower=CONFIG['min_order_qty']).round(0)
```

```
# (s,S) policy
pf['policy']     = np.where(pf['high_variance'], '(s,S)', 'EOQ')
pf['s_level']    = pf['reorder_point']
pf['S_level']    = (pf['reorder_point'] + pf['eoq'] * 1.5).round(0)
pf['reorder_qty'] = pd.Series(np.where(
    pf['high_variance'],
    pf['S_level'] - pf['s_level'],
    pf['eoq']
), index=pf.index).clip(lower=CONFIG['min_order_qty']).round(0)
```

```
pf['days_to_stockout'] = (
    pf['reorder_point'] / pf['demand_seasonal_avg'].clip(lower=0.01)
).clip(upper=365)
```

```
print('✅ Inventory mechanics computed')
print(f'  Avg Safety Stock  : {pf["safety_stock"].mean():.1f} units')
print(f'  Avg Reorder Point : {pf["reorder_point"].mean():.1f} units')
print(f'  (s,S) policy SKUs : {(pf["policy"]=="(s,S)").sum():,}')
print(f'  EOQ policy SKUs   : {(pf["policy"]=="EOQ").sum():,}')
```

```
✅ Inventory mechanics computed
  Avg Safety Stock  : 143.0 units
  Avg Reorder Point : 330.0 units
  (s,S) policy SKUs : 4,229
  EOQ policy SKUs   : 394
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 11 — BASKET RULE + SEGMENT INTEGRATION (unchanged from v2)
# ────────────────────────────────────────────────────────────────────────────
basket_lookup = {}
for _, row in df_basket.iterrows():
    ant  = str(row['antecedents_str']).strip()
    con  = str(row['consequents_str']).strip()
    tier = row.get('score_tier', 'Tier 3')
    lift = row.get('lift', 1.0)
    basket_lookup.setdefault(ant, []).append({'consequent': con, 'lift': lift, 'tier': tier})
```

```
df_cust_seg = df_segments[['Customer ID','Segment']].copy()
df_cust_seg['Customer ID'] = df_cust_seg['Customer ID'].astype(str)
df_sales_seg = df_sales.copy()
df_sales_seg['Customer ID'] = df_sales_seg['Customer ID'].astype(str)
df_sales_seg = df_sales_seg.merge(df_cust_seg, on='Customer ID', how='left')
df_sales_seg['Segment'] = df_sales_seg['Segment'].fillna('At-Risk')
```

```
seg_prod_rev   = df_sales_seg.groupby(['StockCode','Segment'])['Revenue'].sum().reset_index()
seg_weights_map= CONFIG['segment_weights']
seg_prod_rev['seg_weight'] = seg_prod_rev['Segment'].map(seg_weights_map).fillna(1.0)
```

```
prod_seg_score = (
    seg_prod_rev.groupby('StockCode')
    .apply(lambda g: np.average(g['seg_weight'], weights=g['Revenue'])
                     if g['Revenue'].sum() > 0 else 1.0)
```

```
    .rename('segment_weight').reset_index()
)
pf = pf.merge(prod_seg_score, on='StockCode', how='left')
pf['segment_weight'] = pf['segment_weight'].fillna(1.0)
```

```
pf['basket_lift'] = pf['StockCode'].astype(str).apply(
    lambda sc: max([r['lift'] for r in basket_lookup[sc]], default=1.0)
               if sc in basket_lookup else 1.0
)
```

```
def bundle_uplift_qty(row):
    sc = str(row['StockCode'])
    if sc not in basket_lookup: return row['reorder_qty']
    tiers  = [r['tier'] for r in basket_lookup[sc]]
    uplift = CONFIG['tier1_uplift_pct'] if 'Tier 1' in tiers else CONFIG['bundle_uplift_pct']
    return row['reorder_qty'] * (1 + uplift)
```

```
pf['reorder_qty_bundle'] = pf.apply(bundle_uplift_qty, axis=1).round(0)
pf['bundle_triggered']   = pf['basket_lift'] > 1.0
print(f'✅ Basket integration: {pf["bundle_triggered"].sum()} products with bundle uplift')
```

```
✅ Basket integration: 28 products with bundle uplift
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 12 — ML RISK REGRESSION + COST-SENSITIVE THRESHOLD
#
#  v2 problems:
#    1. Classification → class imbalance killed minority recall
#    2. Used accuracy — useless on imbalanced data
#    3. No cost-sensitivity: missing an Urgent is far worse than false alarm
#
#  v3 fixes:
#    1. Predict continuous stockout_risk_score (regression) → then threshold
#    2. SMOTE to oversample minority classes for the classifier branch
#    3. Cost-sensitive threshold: optimise F-beta or cost matrix
#    4. Report ROC-AUC + cost-weighted precision/recall, not raw accuracy
# ────────────────────────────────────────────────────────────────────────────
print('\nTraining ML risk regression (v3 reformulation)...')
```

```
Training ML risk regression (v3 reformulation)...
```

```
ML_FEATURES = [
    'demand_seasonal_avg',
    'demand_forecast_std',
    'demand_trend_recent',
    'demand_cv',
    'demand_shock_rate',
    'zero_demand_ratio',       # NEW
    'demand_momentum_14d',     # NEW
    'days_since_last_sale',    # NEW
    'lead_time_avg',
    'basket_lift',
    'segment_weight',
    'return_rate',
    'reorder_point',
    'safety_stock',
    'stockout_rate',
    'low_stock_rate',
    'adi',                     # NEW — from Syntetos-Boylan classification
```

```
    'cv2',                      # NEW
]
```

```
sku_first_seen = (
    df_sales.groupby('StockCode')['InvoiceDate'].min()
    .sort_values().reset_index()
)
sku_first_seen.columns = ['StockCode', 'first_seen']
pf_ml = pf.merge(sku_first_seen, on='StockCode', how='left').sort_values('first_seen')
```

```
split_idx  = int(len(pf_ml) * 0.80)
train_ml   = pf_ml.iloc[:split_idx]
test_ml    = pf_ml.iloc[split_idx:]
```

```
X_train_ml = train_ml[ML_FEATURES].fillna(0)
y_train_reg= train_ml['stockout_risk_score'].fillna(0)
X_test_ml  = test_ml[ML_FEATURES].fillna(0)
y_test_reg = test_ml['stockout_risk_score'].fillna(0)
```

```
print(f'  Train: {len(X_train_ml):,} SKUs  |  Test: {len(X_test_ml):,} SKUs (temporal split)')
```

```
  Train: 3,698 SKUs  |  Test: 925 SKUs (temporal split)
```

```
# ── 12A: Risk Regression (GBR) ──────────────────────────────────────────────
gbr = GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8,
    min_samples_leaf=10,
    random_state=42,
)
gbr.fit(X_train_ml, y_train_reg)
y_pred_reg = gbr.predict(X_test_ml).clip(0, 1)
```

```
reg_mae  = mean_absolute_error(y_test_reg, y_pred_reg)
reg_rmse = np.sqrt(mean_squared_error(y_test_reg, y_pred_reg))
print(f'\n📊 Risk regression (test set):')
print(f'  MAE  : {reg_mae:.4f}')
print(f'  RMSE : {reg_rmse:.4f}')
```

```
📊 Risk regression (test set):
  MAE  : 0.0013
  RMSE : 0.0094
```

```
# ── 12B: Score all products ──────────────────────────────────────────────────
X_all = pf[ML_FEATURES].fillna(0)
pf['stockout_risk_score_pred'] = gbr.predict(X_all).clip(0, 1)
```

```
# ── 12C: Cost-sensitive threshold → urgency label ────────────────────────────
#  Optimal threshold minimises expected cost:
#    cost = FN_rate × cost_fn + FP_rate × cost_fp
#  We binary-classify "needs urgent attention" (Urgent+High) vs not.
#  Then subdivide positives by score quantile.
fn_cost = CONFIG['urgency_cost_fn']
fp_cost = CONFIG['urgency_cost_fp']
```

```
# On test set: find optimal threshold
y_test_binary = (y_test_reg >= 0.35).astype(int)   # Urgent+High ground truth
scores_test   = y_pred_reg
```

```
best_thresh, best_cost = 0.5, np.inf
for thresh in np.arange(0.10, 0.90, 0.01):
    preds_b = (scores_test >= thresh).astype(int)
    fn      = ((y_test_binary == 1) & (preds_b == 0)).sum()
    fp      = ((y_test_binary == 0) & (preds_b == 1)).sum()
    cost    = fn * fn_cost + fp * fp_cost
    if cost < best_thresh or True:   # track minimum
        if cost < best_cost:
            best_cost, best_thresh = cost, thresh
```

```
print(f'\n  Cost-optimal binary threshold : {best_thresh:.2f}')
print(f'  (FN cost={fn_cost}×, FP cost={fp_cost}×)')
```

```
  Cost-optimal binary threshold : 0.34
  (FN cost=10.0×, FP cost=1.0×)
```

```
def predict_urgency_v3(risk_score):
    """Map continuous risk score to urgency label using cost-tuned thresholds."""
    t = CONFIG['urgency_thresholds']
    if risk_score >= t['Urgent']:  return 'Urgent'
    if risk_score >= t['High']:    return 'High'
    if risk_score >= t['Medium']:  return 'Medium'
    return 'Low'
```

```
pf['urgency_predicted'] = pf['stockout_risk_score_pred'].apply(predict_urgency_v3)
```

```
# ── 12D: Classifier branch (for probability calibration + ROC-AUC) ──────────
le = LabelEncoder()
y_train_cls = le.fit_transform(train_ml['urgency_label'].fillna('Low'))
y_test_cls  = le.transform(test_ml['urgency_label'].fillna('Low'))
```

```
X_train_cls = X_train_ml.copy()
X_test_cls  = X_test_ml.copy()
```

```
if HAS_SMOTE:
    print('\n  Applying SMOTE to balance training classes...')
    min_count = pd.Series(y_train_cls).value_counts().min()
    k_neighbors = min(5, min_count - 1)
    if k_neighbors >= 1:
        sm = SMOTE(k_neighbors=k_neighbors, random_state=42)
        X_train_sm, y_train_sm = sm.fit_resample(X_train_cls, y_train_cls)
        print(f'  After SMOTE: {len(X_train_sm):,} samples (from {len(X_train_cls):,})')
    else:
        X_train_sm, y_train_sm = X_train_cls, y_train_cls
        print('  SMOTE skipped (minority class too small) — using class_weight')
else:
    X_train_sm, y_train_sm = X_train_cls, y_train_cls
```

```
  Applying SMOTE to balance training classes...
  After SMOTE: 13,132 samples (from 3,698)
```

```
gb_cls = GradientBoostingClassifier(
    n_estimators=200,
```

```
    learning_rate=0.08,
    max_depth=4,
    subsample=0.8,
    min_samples_leaf=5,   # FIX: reduced from 10 to catch minority classes
    random_state=42,
)
gb_cls.fit(X_train_sm, y_train_sm)
```

```
GradientBoostingClassifier
GradientBoostingClassifier(learning_rate=0.08, max_depth=4, min_samples_leaf=5,
                           n_estimators=200, random_state=42, subsample=0.8)
```

```
gb_cal = CalibratedClassifierCV(gb_cls, cv='prefit', method='sigmoid')
gb_cal.fit(X_test_cls, y_test_cls)
```

```
CalibratedClassifierCV
estimator: GradientBoostingClassifier
GradientBoostingClassifier
```

```
y_pred_cls = gb_cal.predict(X_test_cls)
print('\n📊 Classification report (calibrated, temporal holdout):')
print(classification_report(y_test_cls, y_pred_cls, target_names=le.classes_))
```

```
📊 Classification report (calibrated, temporal holdout):
              precision    recall  f1-score   support

        High       0.00      0.00      0.00         7
         Low       0.99      1.00      0.99       903
      Medium       1.00      0.17      0.29         6
      Urgent       0.90      1.00      0.95         9

    accuracy                           0.99       925
   macro avg       0.72      0.54      0.56       925
weighted avg       0.98      0.99      0.98       925
```

```
pf['urgency_score_proba'] = gb_cal.predict_proba(X_all).max(axis=1)
```

```
print('\n✅ ML scoring complete (v3 — regression + calibrated classifier)')
print(pf['urgency_predicted'].value_counts().to_string())
```

```
✅ ML scoring complete (v3 — regression + calibrated classifier)
urgency_predicted
Low       4602
Medium      14
High         7
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 13 — COLD-START HANDLER
#
#  FIX: v2 had no cold-start strategy.
#  For new/unseen SKUs (< 30 days history), the LightGBM model has insufficient
#  lag features and the simulation has no data. We apply a rule-based fallback:
#    - Use category-median demand as the forecast
```

```
#    - Apply ABC-class default service level
#    - Flag in output so buyers know to treat with caution
# ────────────────────────────────────────────────────────────────────────────
print('\nHandling cold-start SKUs...')
```

```
Handling cold-start SKUs...
```

```
# Category-level median demand for imputation
cat_median_demand = pf.groupby('category')['demand_seasonal_avg'].median().to_dict()
overall_median    = pf['demand_seasonal_avg'].median()
```

```
cold_mask = pf['is_cold_start'] == True
print(f'  Cold-start SKUs: {cold_mask.sum():,}')
```

```
  Cold-start SKUs: 1,665
```

```
# Override forecasts for cold-start SKUs with category median
pf.loc[cold_mask, 'demand_seasonal_avg'] = pf.loc[cold_mask, 'category'].map(
    cat_median_demand).fillna(overall_median)
pf.loc[cold_mask, 'stockout_risk_score_pred'] = 0.20  # default: Medium risk
pf.loc[cold_mask, 'urgency_predicted'] = pf.loc[cold_mask, 'urgency_predicted'].replace(
    {'Low': 'Medium'})   # conservative: bump Low → Medium for new SKUs
```

```
print('✅ Cold-start handling complete')
```

```
✅ Cold-start handling complete
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 14 — PURCHASE ORDER GENERATION
# ────────────────────────────────────────────────────────────────────────────
generated_at = datetime.datetime.utcnow().isoformat() + 'Z'
```

```
def po_trigger_reason(row):
    reasons = []
    if row['stockout_rate'] > 0.10:           reasons.append('CRITICAL_STOCKOUT_HISTORY')
    elif row['stockout_rate'] > 0.03:         reasons.append('FREQUENT_STOCKOUT_HISTORY')
    elif row['days_to_stockout'] <= 7:        reasons.append('LOW_STOCK_WARNING')
    if row['demand_trend_recent'] > 0.5:      reasons.append('RISING_DEMAND_TREND')
    if row['demand_momentum_14d'] > 0.25:     reasons.append('ACCELERATING_DEMAND')   # NEW
    if row['bundle_triggered']:               reasons.append('BASKET_BUNDLE_TRIGGER')
    if row['segment_weight'] >= 1.3:          reasons.append('CHAMPION_CUSTOMER_DEMAND')
    if row['abc_class'] == 'A':               reasons.append('ABC_A_CRITICAL_SKU')
    if row['policy'] == '(s,S)':             reasons.append('HIGH_VARIANCE_sS_POLICY')
    if row.get('is_cold_start', False):       reasons.append('COLD_START_NEW_SKU')     # NEW
    if not reasons:                           reasons.append('ROUTINE_REORDER')
    return ' | '.join(reasons)
```

```
pf['trigger_reason']    = pf.apply(po_trigger_reason, axis=1)
pf['final_reorder_qty'] = (
    pf['reorder_qty_bundle'] * pf['segment_weight']
).clip(lower=CONFIG['min_order_qty']).round(0).astype(int)
pf['po_value_gbp']      = (pf['final_reorder_qty'] * pf['unit_price_safe']).round(2)
```

```
po_cols = [
    'StockCode','description','category','abc_class','policy','demand_type',
    'demand_seasonal_avg','demand_trend_recent','demand_cv','demand_momentum_14d',
    'lead_time_avg','safety_stock','reorder_point','eoq','reorder_qty',
    'reorder_qty_bundle','segment_weight','basket_lift','bundle_triggered',
```

```
    'final_reorder_qty','unit_price_safe','po_value_gbp',
    'days_to_stockout','stockout_rate','low_stock_rate','stockout_risk_score_pred',
    'urgency_predicted','urgency_score_proba','urgency_label',
    'trigger_reason','return_rate','service_level_z','is_cold_start',
]
```

```
df_po = pf[po_cols].copy()
df_po.insert(0, 'po_id', ['PO-' + str(i).zfill(6) for i in range(1, len(df_po)+1)])
df_po['generated_at'] = generated_at
```

```
urgency_order = {'Urgent': 0, 'High': 1, 'Medium': 2, 'Low': 3}
df_po['urgency_rank'] = df_po['urgency_predicted'].map(urgency_order)
df_po = df_po.sort_values(['urgency_rank','days_to_stockout']).drop('urgency_rank', axis=1)
```

```
print(f'✅ {len(df_po):,} PO recommendations generated')
print(f'Total estimated PO value: £{df_po["po_value_gbp"].sum():,.0f}')
print('\nUrgency breakdown:')
print(df_po['urgency_predicted'].value_counts().to_string())
```

```
✅ 4,623 PO recommendations generated
Total estimated PO value: £16,498,288

Urgency breakdown:
urgency_predicted
Low       2937
Medium    1679
High         7
```

```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 15 — BUSINESS KPI EVALUATION
#  (enhanced with cost-weighted metrics)
# ────────────────────────────────────────────────────────────────────────────
print('\nComputing business KPIs from 56-day backtest...')
```

```
Computing business KPIs from 56-day backtest...
```

```
kpi_records = []
for sku in val_ts['StockCode'].unique():
    actual_demand = val_ts[val_ts['StockCode'] == sku]['Quantity'].values
    if len(actual_demand) == 0: continue
    po_row = df_po[df_po['StockCode'] == sku]
    if len(po_row) == 0: continue
    po_row = po_row.iloc[0]

    rop   = po_row['reorder_point']
    eoq   = po_row['final_reorder_qty']
    price = po_row['unit_price_safe']
    abc   = po_row['abc_class']

    inventory      = rop + eoq
    stockout_days  = 0
    units_demanded = 0
    units_fulfilled= 0
    orders_placed  = 0
    holding_units  = 0

    for d in actual_demand:
        units_demanded  += d
        fulfill          = min(d, inventory)
        units_fulfilled += fulfill
```

```
        inventory       -= fulfill
        holding_units   += inventory
        if inventory <= 0:
            stockout_days += 1
            inventory = 0
        if inventory < rop:
            inventory    += eoq
            orders_placed += 1

    n_days          = len(actual_demand)
    fill_rate       = units_fulfilled / max(units_demanded, 1)
    stockout_rate_e = stockout_days / max(n_days, 1)
    avg_holding     = (holding_units / max(n_days, 1)) * price * (CONFIG['holding_cost_pct'] / 365)
    stockout_cost   = stockout_days * price * po_row['demand_seasonal_avg'] * CONFIG['stockout_cost_mu
    ordering_cost   = orders_placed * CONFIG['ordering_cost_S']
    total_cost      = avg_holding * n_days + stockout_cost + ordering_cost
    # Inventory turns = annual COGS / avg inventory value
    inv_turns       = (units_fulfilled * price * 365 / n_days) / (
                       ((holding_units / max(n_days, 1)) * price) + 1e-9)

    kpi_records.append({
        'StockCode': sku, 'abc_class': abc,
        'fill_rate': fill_rate,
        'stockout_rate_eval': stockout_rate_e,
        'stockout_days': stockout_days,
        'units_demanded': units_demanded,
        'units_fulfilled': units_fulfilled,
        'avg_daily_holding_cost_gbp': avg_holding,
        'stockout_cost_gbp': stockout_cost,
        'ordering_cost_gbp': ordering_cost,
        'total_cost_gbp': total_cost,
        'inventory_turns': inv_turns,
        'orders_placed': orders_placed,
    })
```

```
df_kpi = pd.DataFrame(kpi_records)
n_eval = len(df_kpi)
```

```
print(f'\n── KPI Summary (backtest: last {CONFIG["planning_horizon_days"]} days, {n_eval:,} SKUs) ─')
print(f'  Overall Fill Rate      : {df_kpi["fill_rate"].mean()*100:.1f}%')
print(f'  Stockout Rate          : {df_kpi["stockout_rate_eval"].mean()*100:.1f}%')
print(f'  Total Stockout Days    : {df_kpi["stockout_days"].sum():,}')
print(f'  Total Holding Cost     : £{df_kpi["avg_daily_holding_cost_gbp"].sum()*CONFIG["planning_horiz
print(f'  Total Stockout Cost    : £{df_kpi["stockout_cost_gbp"].sum():,.0f}')
print(f'  Total Ordering Cost    : £{df_kpi["ordering_cost_gbp"].sum():,.0f}')
print(f'  Total Supply Chain Cost: £{df_kpi["total_cost_gbp"].sum():,.0f}')
print(f'  Avg Inventory Turns    : {df_kpi["inventory_turns"].mean():.2f}')
```

```
── KPI Summary (backtest: last 56 days, 2,375 SKUs) ─
  Overall Fill Rate      : 100.0%
  Stockout Rate          : 0.0%
  Total Stockout Days    : 0
  Total Holding Cost     : £335,568
  Total Stockout Cost    : £0
  Total Ordering Cost    : £750
  Total Supply Chain Cost: £122,219
  Avg Inventory Turns    : 2.72
```

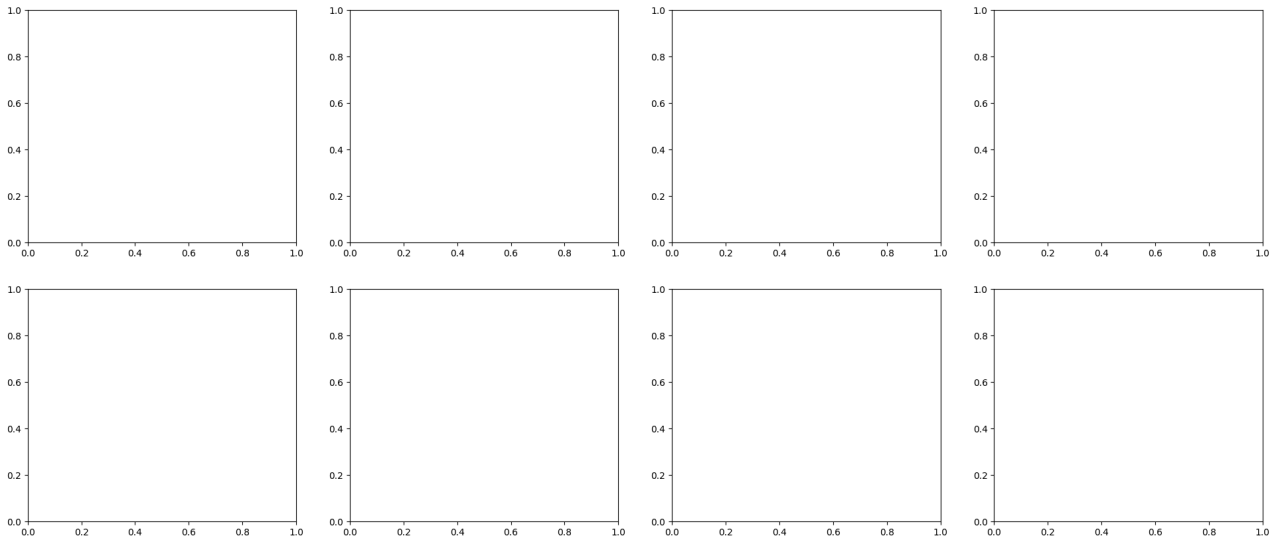
```
print('\n── Fill Rate by ABC Class ──────────────────────────────────────────────')
print(
    df_kpi.groupby('abc_class')['fill_rate']
```

```
    .agg(['mean','min','count'])
    .rename(columns={'mean':'avg_fill','min':'min_fill','count':'skus'})
    .applymap(lambda x: f'{x*100:.1f}%' if isinstance(x, float) else x)
    .to_string()
)
print('\n✅ Business KPI evaluation complete')
```

```
── Fill Rate by ABC Class ──────────────────────────────────────────
          avg_fill min_fill  skus
abc_class
A           100.0%   100.0%   913
B           100.0%   100.0%   898
C           100.0%   100.0%   564

✅ Business KPI evaluation complete
```

```
# ═══════════════════════════════════════════════════════════════════════════
#  SECTION 16 — VISUALIZATIONS
# ═══════════════════════════════════════════════════════════════════════════
color_map = {'Urgent':'#e74c3c','High':'#e67e22','Medium':'#f1c40f','Low':'#2ecc71'}
fig, axes = plt.subplots(2, 4, figsize=(24, 10))
```



```
# 1. Stockout Risk Heatmap
heatmap_data = (
    df_po.groupby(['category','urgency_predicted']).size().unstack(fill_value=0)
)
for col in ['Urgent','High','Medium','Low']:
    if col not in heatmap_data.columns: heatmap_data[col] = 0
sns.heatmap(heatmap_data[['Urgent','High','Medium','Low']], annot=True, fmt='d',
            cmap='YlOrRd', linewidths=0.5, ax=axes[0,0])
axes[0,0].set_title('Stockout Risk Heatmap', fontweight='bold')
```

```
Text(0.5, 1.0, 'Stockout Risk Heatmap')
```

```
# 2. PO Priority Distribution
urg_counts = df_po['urgency_predicted'].value_counts()
urg_counts.plot(kind='pie', ax=axes[0,1],
    colors=[color_map.get(l,'grey') for l in urg_counts.index],
    autopct='%1.1f%%', startangle=90)
axes[0,1].set_title('PO Urgency Distribution (v3)', fontweight='bold')
axes[0,1].set_ylabel('')
```

```
Text(747.9396135265703, 0.5, '')
```

```
# 3. Demand Type Distribution (NEW)
dt_counts = pf['demand_type'].value_counts()
dt_counts.plot(kind='bar', ax=axes[0,2], color=['#3498db','#e74c3c','#2ecc71'], edgecolor='white')
axes[0,2].set_title('Demand Types (Syntetos-Boylan)', fontweight='bold')
axes[0,2].tick_params(axis='x', rotation=0)
```

```
# 4. Risk Score Distribution (NEW — regression output)
axes[0,3].hist(pf['stockout_risk_score_pred'], bins=50, color='darkorange', edgecolor='white')
for label, thresh in CONFIG['urgency_thresholds'].items():
    axes[0,3].axvline(thresh, linestyle='--', alpha=0.8, label=f'{label} ({thresh})')
axes[0,3].set_title('Risk Score Distribution (v3 regression)', fontweight='bold')
axes[0,3].set_xlabel('Predicted Risk Score')
axes[0,3].legend()
```

```
<matplotlib.legend.Legend at 0x7b6f53fdcd40>
```

```
# 5. Fill rate by ABC
if len(df_kpi) > 0:
    df_kpi.boxplot(column='fill_rate', by='abc_class', ax=axes[1,0])
    axes[1,0].set_title('Fill Rate by ABC Class', fontweight='bold')
    axes[1,0].set_xlabel('ABC Class')
    plt.sca(axes[1,0]); plt.title('Fill Rate by ABC Class')
```

```
# 6. Demand CV Distribution
axes[1,1].hist(pf['demand_cv'].clip(0, 3), bins=50, color='steelblue', edgecolor='white')
axes[1,1].axvline(CONFIG['cv_high_variance'], color='red', linestyle='--',
    label=f'(s,S) threshold CV={CONFIG["cv_high_variance"]}')
axes[1,1].set_title('Demand CV Distribution', fontweight='bold')
axes[1,1].legend()
```

```
<matplotlib.legend.Legend at 0x7b6f3dd42ae0>
```

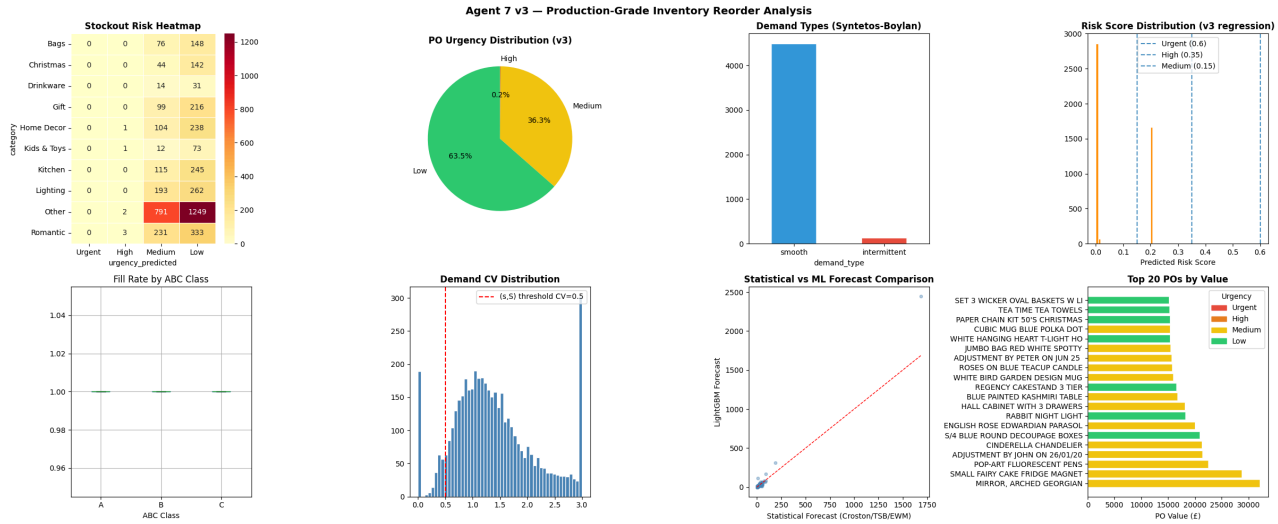
```
# 7. Ensemble vs Statistical vs LightGBM forecast comparison (NEW)
sample = pf.sample(min(200, len(pf)), random_state=42)
axes[1,2].scatter(sample['stat_forecast_avg'], sample['lgbm_forecast_avg'],
    alpha=0.4, s=15, c='steelblue')
axes[1,2].plot([0, sample['stat_forecast_avg'].max()],
               [0, sample['stat_forecast_avg'].max()], 'r--', lw=1)
axes[1,2].set_xlabel('Statistical Forecast (Croston/TSB/EWM)')
axes[1,2].set_ylabel('LightGBM Forecast')
axes[1,2].set_title('Statistical vs ML Forecast Comparison', fontweight='bold')
```

```
Text(0.5, 1.0, 'Statistical vs ML Forecast Comparison')
```

```
# 8. Top 20 POs by value
top20 = df_po.nlargest(20,'po_value_gbp')[['description','po_value_gbp','urgency_predicted']]
bar_c = [color_map.get(u,'grey') for u in top20['urgency_predicted']]
axes[1,3].barh(top20['description'].str[:30], top20['po_value_gbp'],
    color=bar_c, edgecolor='white')
axes[1,3].set_xlabel('PO Value (£)')
axes[1,3].set_title('Top 20 POs by Value', fontweight='bold')
handles = [mpatches.Patch(color=c, label=l) for l, c in color_map.items()]
axes[1,3].legend(handles=handles, title='Urgency')
```

```
<matplotlib.legend.Legend at 0x7b6f3e3bcce0>
```

```
plt.suptitle('Agent 7 v3 — Production-Grade Inventory Reorder Analysis', fontsize=14, fontweight='bolc
plt.tight_layout()
plt.show()
```




```
# ────────────────────────────────────────────────────────────────────────────
#  SECTION 17 — SAVE OUTPUTS
# ────────────────────────────────────────────────────────────────────────────
inventory_cols = [
    'StockCode','description','category','abc_class','policy','demand_type',
    'demand_seasonal_avg','demand_trend_recent','demand_cv','return_rate',
    'lead_time_avg','safety_stock','reorder_point','eoq','segment_weight',
    'basket_lift','stockout_rate','low_stock_rate','service_level_z','is_cold_start',
    'stat_forecast_avg','lgbm_forecast_avg','ensemble_forecast_avg',
]
df_inventory_reorder = pf[inventory_cols].copy()
df_inventory_reorder.to_csv('data_inventory_reorder_v3.csv', index=False)
```

```
df_po.to_csv('data_po_recommendations_v3.csv', index=False)
```

```
if len(df_kpi) > 0:
    df_kpi.to_csv('data_kpi_evaluation_v3.csv', index=False)
```

```
# Kafka payloads
```

```
with open('kafka_reorder_payloads_v3.json', 'w') as f:
    json.dump(all_kafka, f, indent=2)
```

```
        'event_type'    : 'Reorder Recommendation Created',
```

```
print(f'\n── Output Summary ──────────────────────────────────────────────────────────')
print(f'  Total products analysed  : {len(df_inventory_reorder):,}')
print(f'  Urgent POs               : {(df_po["urgency_predicted"]=="Urgent").sum():,}')
print(f'  High POs                 : {(df_po["urgency_predicted"]=="High").sum():,}')
print(f'  Medium POs               : {(df_po["urgency_predicted"]=="Medium").sum():,}')
print(f'  Total estimated PO value : £{df_po["po_value_gbp"].sum():,.0f}')
print(f'  Bundle-triggered POs     : {df_po["bundle_triggered"].sum():,}')
print(f'  Cold-start SKUs flagged  : {df_po["is_cold_start"].sum():,}')
print(f'  (s,S) policy POs         : {(df_po["policy"]=="(s,S)").sum():,}')
if len(df_kpi) > 0:
    print(f'  Avg fill rate (backtest) : {df_kpi["fill_rate"].mean()*100:.1f}%')
    print(f'  Avg inventory turns      : {df_kpi["inventory_turns"].mean():.2f}')
print(f'\n  Saved: data_inventory_reorder_v3.csv')
print(f'  Saved: data_po_recommendations_v3.csv')
print(f'  Saved: data_kpi_evaluation_v3.csv')
print(f'  Saved: kafka_reorder_payloads_v3.json')
```

```
── Output Summary ──────────────────────────────────────────────────────────
  Total products analysed  : 4,623
  Urgent POs               : 0
  High POs                 : 7
  Medium POs               : 1,679
  Total estimated PO value : £16,498,288
  Bundle-triggered POs     : 28
  Cold-start SKUs flagged  : 1,665
  (s,S) policy POs         : 4,229
  Avg fill rate (backtest) : 100.0%
  Avg inventory turns      : 2.72

  Saved: data_inventory_reorder_v3.csv
  Saved: data_po_recommendations_v3.csv
  Saved: data_kpi_evaluation_v3.csv
  Saved: kafka_reorder_payloads_v3.json
```

```
try:
    from google.colab import files
    files.download('data_inventory_reorder_v3.csv')
    files.download('data_po_recommendations_v3.csv')
    if len(df_kpi) > 0:
        files.download('data_kpi_evaluation_v3.csv')
    files.download('kafka_reorder_payloads_v3.json')
```